VectorQuant — AI Agent Context Window (Claude/Gemini)

This file serves as a persistent context window for AI assistants to understand the project's unique philosophy, architecture, and constraints.

🌟 Project Philosophy: The Zero-Dependency Engine

VectorQuant is a self-contained quantitative finance battery. Its mission is to be the **standard for zero-dependency high-performance finance** in Python. No NumPy, No SciPy, No BLAS/LAPACK.

System Architecture

```
Parse error on line 2:
...bgraph Python_Layer ["Python Layer (High
----------------------^
```

```
Expecting 'SEMI', 'NEWLINE', 'SPACE', 'EOF', 'GRAPH', 'DIR',
'TAGEND', 'TAGSTART', 'UP', 'DOWN', 'subgraph', 'end', 'SQE',
'PE', '-)', 'DIAMOND_STOP', 'MINUS', '--', 'ARROW_POINT',
'ARROW_CIRCLE', 'ARROW_CROSS', 'ARROW_OPEN',
'DOTTED_ARROW_POINT', 'DOTTED_ARROW_CIRCLE',
'DOTTED_ARROW_CROSS', 'DOTTED_ARROW_OPEN', '==',
'THICK_ARROW_POINT', 'THICK_ARROW_CIRCLE', 'THICK_ARROW_CROSS',
'THICK_ARROW_OPEN', 'PIPE', 'STYLE', 'LINKSTYLE', 'CLASSDEF',
'CLASS', 'CLICK', 'DEFAULT', 'NUM', 'PCT', 'COMMA', 'ALPHA',
'COLON', 'BRKT', 'DOT', 'PUNCTUATION', 'UNICODE_TEXT', 'PLUS',
'EQUALS', 'MULT', got 'SQS'
```

🎯 Performance Goals (Achieved vs Target)

- **Matrix Multiplication**: Achieved **165x** speedup; Cache-blocked kernels with **AVX2 SIMD**.
- **Covariance**: Achieved **258x** speedup; Highly optimized OpenMP column-wise parallelization.
- **Monte Carlo**: Achieved **2.3x** speedup relative to Numba JIT; 100% C-native stochastic loops.
- **Latency**: Sub-millisecond Python ↔ C boundary overhead using contiguous flat-buffer transfers.

🔢 Core Mathematical Foundations

All advanced quant models in VectorQuant are built on these internal primitives:

1. **Matrix Multiplication (ikj)**: The backbone of Covariance, Regression, and SVD.
2. **Xoroshiro128+ PRNG**: High-performance, statistically strong RNG with jumping support.
3. **Internal Solvers**: High-performance LU, Cholesky, QR, Eigen, and SVD (Zero-Dependency).
4. **BFGS Optimizer**: Standard Quasi-Newton path for portfolio optimization and calibration.

🛠 Strategic Architecture Rules

1. Backend Dispatch Logic

Capability detection occurs strictly at import time:

- **C Extensions available?** → Load `active_backend = CBackend()`.
- **C Extensions missing?** → Load `active_backend = PythonBackend()` (with optional Numba). *Rule: Do not add redundant `if C_AVAILABLE` checks inside tight mathematical loops.*

2. Memory Model

AI agents must maintain this layout for all new kernels:

- **SIMD Optimized**: Kernels use **AVX2/SSE** auto-vectorization (`/arch:AVX2` on Windows).
- **Cache-Blocked**: Large matrix operations use size-conditional blocking for L1/L2 hits.

3. Parallelization Strategy (OpenMP)

Parallelism occurs at the **Outer Level** of computation:

- **Monte Carlo**: Parallelize over *simulation paths*.
- **Covariance**: Parallelize over *columns*.
- **Matrix Multiply**: Parallelize over *rows*.

4. Deterministic AI Verification

VectorQuant is built for **AI Reasoning**. Calculations must support:

- **Reproducible results**: Seeded PRNGs only.
- **Proof Traces**: Intermediate symbolic stems for AI consistency checking.
- **Sensitivities**: Native Autodiff (Dual Numbers) for second-order Greeks/sensitivity analysis.

🗺️ Future Roadmap

Phase 7 & 7.5: Deep Optimization (Completed)

- [x] Internal Linear Algebra Solvers: LU, SVD, Cholesky, QR, Eigen (Zero-Dependency).
- [x] SIMD Integration: AVX2/SSE compiler-level vectorization enabled.
- [x] Cache-Blocked MatMul: Size-conditional blocking for large matrix operations.
- [x] Parallel Monte Carlo: Native Box-Muller & Parallel simulation loops.

Phase 8: Advanced Mathematical Models

- [ ] **Streaming Algos**: Incremental statistics O(1) for real-time risk tracking.
- [ ] **Batched Linalg**: Batched LU, QR, and SVD for high-throughput processing.
- [ ] **Kalman Filters**: C-accelerated state-space models for time-series.
- [ ] **Sparse Matrix Engine**: Internal CSR paths for high-dimensional models.
- [ ] **Advanced Sobol**: Scrambled Halton/Sobol for QMC.

Phase 9: Distributed & Control

- **GPU Integration**: Targeted CUDA kernels for massive MC.
- **Stochastic Control**: HJB solvers and Reinforcement Learning hooks.

🧠 AI Agent Implementation Checklist

1. **Zero External Dependencies**: Never suggest `import numpy`. Re-implement the kernel in C.
2. **Internal Optimization**: Check `linalg.c` before suggesting new math.
3. **MSVC Compatibility**: Use C89 loop declarations for OpenMP (`int i;` before `for`).
4. **High-Value Kernels**: Prioritize performance optimization for MatMul, Covariance, and RNG.